

CONTEXT.md: The Missing Layer of the Repository

A Manifesto for Agents That Consult Before They Code

Pavel Kerbel

Independent Researcher

Abstract. Repositories version source code, tests, documentation, and configuration. None of them versions what the project *knows*. The decisions, the rejected alternatives, the constraint discovered at two in the morning — the context that makes a senior engineer senior — lives in chat scrollback and in people's heads, and every agent session starts without it. This note proposes a new repository primitive: the *Repository Context Layer*, a version-controlled, human-readable store that acts as the authoritative operating context for AI agents. The lifecycle: consult before planning, execute, update the context, commit both together. Git stores what changed; Repository Context stores what the project knows. Everything else is decoration.

Index Terms. repository context layer, context.md, agentic software development, institutional knowledge, decision records, invariant constraints, self-improving repositories, consult-before-plan.

I. AGENTS WITH AMNESIA

An agent session begins with a strong model and an empty head. It can read every file in the repository and still not know why any of them look the way they do. Code records outcomes with the reasons deleted: a set of answers, the questions thrown away.

So the agent re-litigates every settled argument. It proposes the ORM you removed in March. It reintroduces the abstraction that failed in production. None of this is a model problem: the knowledge it needed exists, just nowhere a model can reach. The most expensive knowledge in a repository is negative knowledge — what was tried and rejected — and no standard artifact holds it. The result: repeated mistakes, architectural drift, and a permanent prompt-engineering tax — re-explaining the same project to the same model, forever.

II. THE MISSING LAYER

The fix is not a better prompt or a bigger context window. It is a missing layer of the repository itself.

A Repository Context Layer is a version-controlled, human-readable context store that lives alongside the codebase and acts as the authoritative operating context for AI agents working in it.

Git stores what changed.

Repository Context stores what the project knows.

Call it context, not memory. Memory is personal, fuzzy, optional; it evaporates with the session that formed it. Context is the deterministic control plane for the model's runtime behavior: on disk, versioned with the code, at a known address. It stops being a pile of notes the moment the agent is *required* to consult it and *expected* to maintain it.

III. THE LIFECYCLE

consult → execute → update context → commit

Before planning, the agent reads the context layer. A plan made without priors is a guess with good formatting. Then it executes. Then, the step almost every system skips: before committing, the agent writes back what execution taught it — the package that breaks the ARM64 build, the timeout a downstream proxy silently enforces.

The commit carries both the code and the sharpened context. No external store can offer this: context evolves with the repository because it travels with it — branching when the code branches, merging when it merges, rolling back when it rolls back.

IV. DESIGN PRINCIPLES

Git-native. Versioned by the tool that already solved provenance, review, rollback, and blame. If you cannot `git log` your agent's beliefs, you cannot debug them.

Human-readable. Plain Markdown, readable in review, editable in any editor.

Reviewable. When an agent changes its own operating rules, the change appears in the diff next to the code that motivated it, and the human ap-

proves both in one gesture — self-modification with a human veto. An agent that edits its context in the open is safer than one that remembers in the dark.

Branch-aware. Context follows the branching model for free; no second source of truth to reconcile.

Tool-independent. No SDK, no server, no vendor: any agent that can read a file participates, and any human with a text editor is a first-class writer.

Incrementally evolving. Start with three sections and one honest sentence in each; the layer grows when the project learns, not when a template demands it.

Deterministic discovery. The agent must find the context without searching for it. A fixed path is an API.

V. MINIMAL SPECIFICATION

Discovery order: `.repo/context.md`, then `context.md` at the repository root. First hit wins. Three sections are required:

Intent — what this project is, and the design philosophy everything else must serve. **Constraints** — the non-negotiable rules, each with its reason. A rule without a reason is a superstition: the agent will comply but can never generalize. Record the rejection, not just the rule. **Evolved Context** — an append-only ledger of what agents and humans learned while working here. Entries that prove out graduate into Constraints; that reviewed promotion is the self-improvement loop made tangible.

```
# Repository Context

## Intent
Local-first CLI. Files are the source of
truth; SQLite is a rebuildable index.

## Constraints
- No ORM. Rejected 2026-03: query opacity
  broke the offline repair path.

## Evolved Context
- [2026-06-29] pkg X >=3.0 breaks ARM64
  builds. Pin to 2.3.x until fixed.
```

That is the whole specification: three headers and a discovery path. Everything beyond it — decision logs, pattern catalogs, per-directory context — is convention layered on top. Do not over-specify. Under-specified and adopted beats complete and ignored.

VI. WHY THIS MATTERS

Repositories are about to change population: soon most readers and writers of a codebase will not be human. The repository of the next decade contains source code, tests, documentation, and context — a first-class artifact, reviewed, versioned, and trusted like code.

The deeper shift is where improvement accumulates. Model weights are frozen between releases; repositories do not have to be. With a context layer, every session ends with sharper priors than it started, compounding at merge speed rather than training speed. The repository gets smarter, not the model. Self-improvement, done this way, is a merge, not a fine-tune — and the unit of learning is a reviewed line in `context.md`.

VII. A REFERENCE IMPLEMENTATION

Metatron is one implementation of this layer: decisions and project facts kept as git-backed Markdown, served to agents at consult time, feedback routed into the evolved-context ledger; an optional index accelerates retrieval, but files remain the truth. It is an implementation, not the architecture — any tool, or none at all, can implement the layer. The abstraction should outlive every product built on it, including this one.